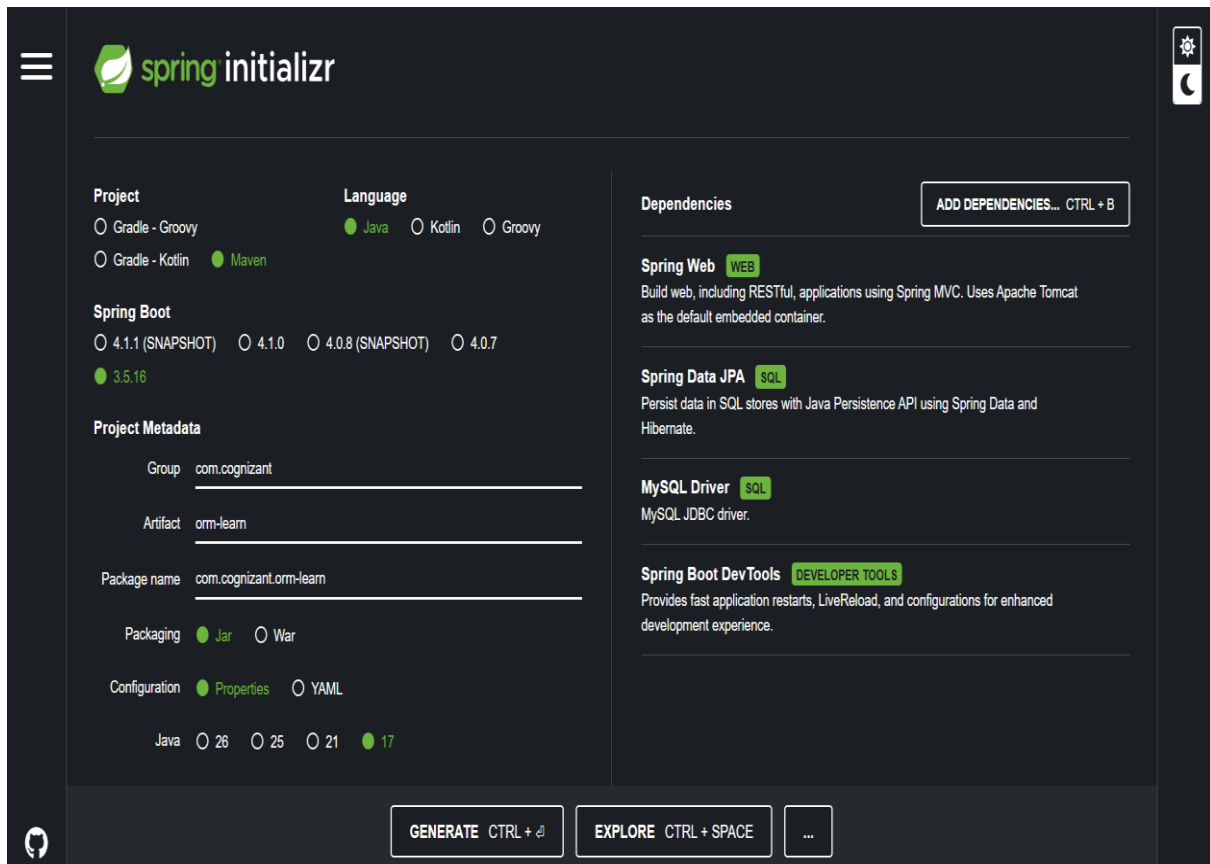


Spring Data JPA - Quick Example



The image shows the Spring Initializr web interface. It is a dark-themed form for generating a Spring project. The interface is divided into several sections: Project, Language, Spring Boot, Project Metadata, Dependencies, and a bottom bar with action buttons.

Project

- ☐ Gradle - Groovy
- ☐ Gradle - Kotlin
- ☒ Maven

Language

- ☒ Java
- ☐ Kotlin
- ☐ Groovy

Spring Boot

- ☐ 4.1.1 (SNAPSHOT)
- ☐ 4.1.0
- ☐ 4.0.8 (SNAPSHOT)
- ☐ 4.0.7
- ☒ 3.5.16

Project Metadata

Group:

Artifact:

Package name:

Packaging: ☒ Jar ☐ War

Configuration: ☒ Properties ☐ YAML

Java: ☐ 26 ☐ 25 ☐ 21 ☒ 17

Dependencies

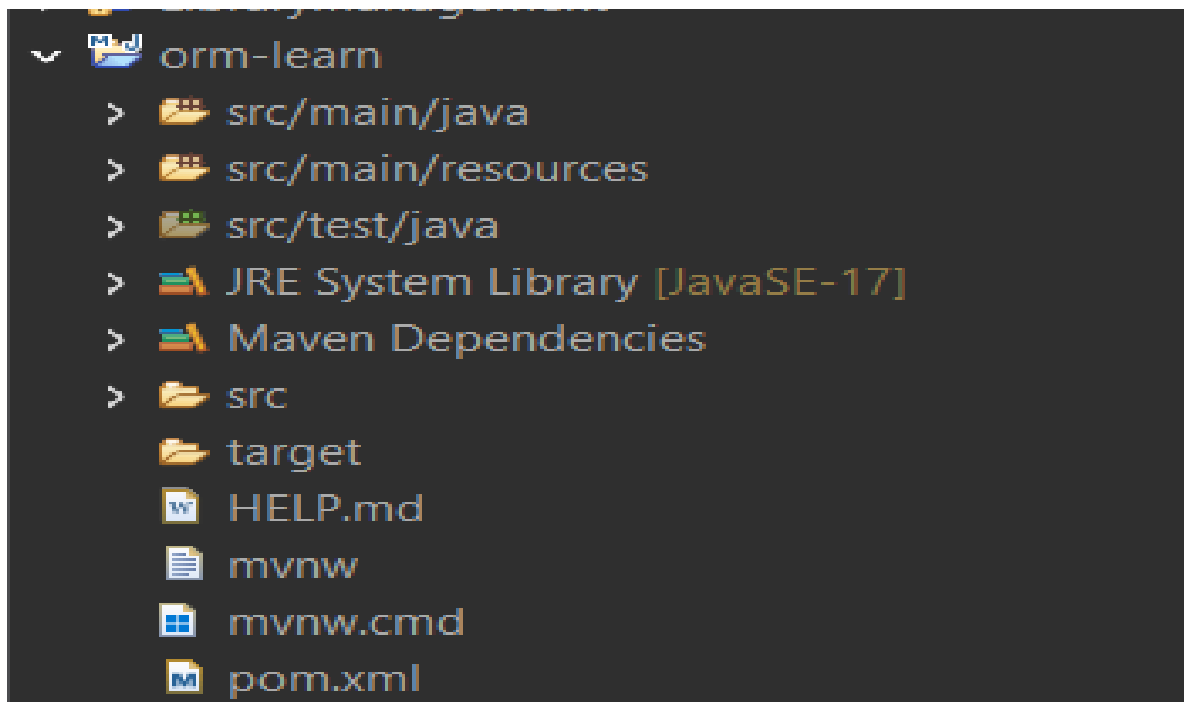
Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

MySQL Driver SQL
MySQL JDBC driver.

Spring Boot DevTools DEVELOPER TOOLS
Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Buttons: GENERATE (CTRL + G), EXPLORE (CTRL + SPACE), ...



[illegible]

Navigator

SCHEMAS

Filter objects

airline_management

deepika

movies_db

omlearn

qspiders

sakila

school

securin

sys

ticket_booking

ticket_booking_system

world

Administration

Schemas

Information

No object selected

Query 1

Limit to 1000 rows

1 • INSERT INTO country VALUES ('IN', 'India');

2

3 • INSERT INTO country VALUES ('US', 'United States');

Output

Action Output

#	Time	Action
✓ 1	18:18:02	CREATE DATABASE omlearn
✓ 2	18:19:09	use omlearn
✓ 3	18:20:09	CREATE TABLE country (code VARCHAR(2) PRIMARY KEY, name VARCHAR(50))
✓ 4	18:20:57	INSERT INTO country VALUES ('IN', 'India')
✓ 5	18:20:57	INSERT INTO country VALUES ('US', 'United States')

1 • `SELECT * FROM country;`

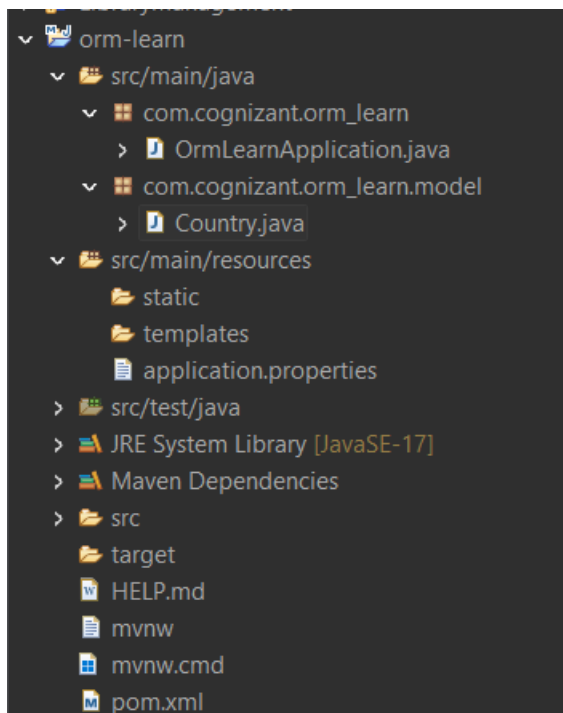
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content

	code	name
▶	IN	India
	US	United States
*	NULL	NULL

```

1spring.application.name=orm-learn
2
3# Spring Framework logs
4logging.level.org.springframework=info
5logging.level.com.cognizant=debug
6
7# Hibernate logs
8logging.level.org.hibernate.SQL=trace
9logging.level.org.hibernate.orm.jdbc.bind=trace
10
11# Database configuration
12spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
13spring.datasource.username=root
14spring.datasource.password=Mncvv2005*
15
16# Hibernate configuration
17spring.jpa.hibernate.ddl-auto=validate
18spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
19
20spring.jpa.show-sql=true
21spring.jpa.properties.hibernate.format_sql=true

```



```
1 package com.cognizant.orm_learn.model;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.Id;
6 import jakarta.persistence.Table;
7
8 @Entity
9 @Table(name = "country")
10 public class Country {
11
12     @Id
13     @Column(name = "code")
14     private String code;
15
16     @Column(name = "name")
17     private String name;
18
19     public Country() {
20     }
21
22     public String getCode() {
23         return code;
24     }
25
26     public void setCode(String code) {
27         this.code = code;
28     }
29
30     public String getName() {
31         return name;
32     }
33 }
```

```

33
34     public void setName(String name) {
35         this.name = name;
36     }
37
38     @Override
39     public String toString() {
40         return "Country [code=" + code + ", name=" + name + "]";
41     }
42 }

```

```

v orm-learn
  v src/main/java
    v com.cognizant.orm_learn
      > OrmLearnApplication.java
    v com.cognizant.orm_learn.model
      > Country.java
    v com.cognizant.orm_learn.repository
      > CountryRepository.java
    v com.cognizant.orm_learn.service
      > CountryService.java
  v src/main/resources
    static
    templates
    application.properties
  > src/test/java
  > JRE System Library [JavaSE-17]
  > Maven Dependencies
  > src
  target
  HELP.md
  mvnw
  mvnw.cmd
  pom.xml

```

```
1 package com.cognizant.orm_learn.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5
6 import com.cognizant.orm_learn.model.Country;
7
8 @Repository
9 public interface CountryRepository extends JpaRepository<Country, String> {
10
11 }
```

```
1 package com.cognizant.orm_learn.service;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7 import org.springframework.transaction.annotation.Transactional;
8
9 import com.cognizant.orm_learn.model.Country;
10 import com.cognizant.orm_learn.repository.CountryRepository;
11
12 @Service
13 public class CountryService {
14
15     @Autowired
16     private CountryRepository countryRepository;
17
18     @Transactional
19     public List<Country> getAllCountries() {
20
21         return countryRepository.findAll();
22
23     }
24
25 }
```

```

1 package com.cognizant.orm_learn;
2
3 import java.util.List;
4
5 import org.slf4j.Logger;
6 import org.slf4j.LoggerFactory;
7 import org.springframework.boot.SpringApplication;
8 import org.springframework.boot.autoconfigure.SpringBootApplication;
9 import org.springframework.context.ApplicationContext;
10
11 import com.cognizant.orm_learn.model.Country;
12 import com.cognizant.orm_learn.service.CountryService;
13
14 @SpringBootApplication
15 public class OrmLearnApplication {
16
17     private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);
18
19     private static CountryService countryService;
20
21     public static void main(String[] args) {
22
23         ApplicationContext context = SpringApplication.run(OrmLearnApplication.class, args);
24
25         countryService = context.getBean(CountryService.class);
26
27         testGetAllCountries();
28
29     }
30

```

```

31     private static void testGetAllCountries() {
32
33         LOGGER.info("Start");
34
35         List<Country> countries = countryService.getAllCountries();
36
37         LOGGER.debug("Countries = {}", countries);
38
39         LOGGER.info("End");
40
41     }
42
43 }

```

```
16 private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);
17

Console X
OrmLearnApplication [Java Application] C:\Program Files\Java\jdk-17\bin\java.exe (1 Jul 2026, 7:07:34 pm) [pid: 11868]

:: Spring Boot :: (v3.5.16)

2026-07-01T19:07:36.043+05:30 INFO 11868 --- [orm-learn] [ restartedMain] c.c.orm_learn.OrmLearnApplication
2026-07-01T19:07:36.050+05:30 DEBUG 11868 --- [orm-learn] [ restartedMain] c.c.orm_learn.OrmLearnApplication
2026-07-01T19:07:36.051+05:30 INFO 11868 --- [orm-learn] [ restartedMain] c.c.orm_learn.OrmLearnApplication
2026-07-01T19:07:36.137+05:30 INFO 11868 --- [orm-learn] [ restartedMain] .e.DevToolsPropertyDefaultsPosti
2026-07-01T19:07:36.138+05:30 INFO 11868 --- [orm-learn] [ restartedMain] .e.DevToolsPropertyDefaultsPosti
2026-07-01T19:07:37.045+05:30 INFO 11868 --- [orm-learn] [ restartedMain] .s.d.r.c.RepositoryConfiguratio
2026-07-01T19:07:37.121+05:30 INFO 11868 --- [orm-learn] [ restartedMain] .s.d.r.c.RepositoryConfiguratio
2026-07-01T19:07:37.896+05:30 INFO 11868 --- [orm-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatW
2026-07-01T19:07:37.916+05:30 INFO 11868 --- [orm-learn] [ restartedMain] o.apache.catalina.core.Standard
2026-07-01T19:07:37.917+05:30 INFO 11868 --- [orm-learn] [ restartedMain] o.apache.catalina.core.Standard
2026-07-01T19:07:37.977+05:30 INFO 11868 --- [orm-learn] [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].
2026-07-01T19:07:37.980+05:30 INFO 11868 --- [orm-learn] [ restartedMain] w.s.c.ServletWebServerApplicati
2026-07-01T19:07:38.291+05:30 INFO 11868 --- [orm-learn] [ restartedMain] o.hibernate.jpa.internal.util.L
2026-07-01T19:07:38.383+05:30 INFO 11868 --- [orm-learn] [ restartedMain] org.hibernate.Version
2026-07-01T19:07:38.423+05:30 INFO 11868 --- [orm-learn] [ restartedMain] o.h.c.internal.RegionFactoryIni
2026-07-01T19:07:38.753+05:30 INFO 11868 --- [orm-learn] [ restartedMain] o.s.o.j.p.SpringPersistenceUnit
2026-07-01T19:07:38.825+05:30 INFO 11868 --- [orm-learn] [ restartedMain] com.zaxxer.hikari.HikariDataSou
2026-07-01T19:07:39.148+05:30 INFO 11868 --- [orm-learn] [ restartedMain] com.zaxxer.hikari.pool.HikariPo
2026-07-01T19:07:39.157+05:30 INFO 11868 --- [orm-learn] [ restartedMain] com.zaxxer.hikari.HikariDataSou
2026-07-01T19:07:39.246+05:30 INFO 11868 --- [orm-learn] [ restartedMain] org.hibernate.orm.connections.pr
Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
```

```
2026-07-01T19:07:39.157+05:30 INFO 11868 --- [orm-learn] [ restartedMain] com.zaxxer.hikari.HikariDataSou
2026-07-01T19:07:39.246+05:30 INFO 11868 --- [orm-learn] [ restartedMain] org.hibernate.orm.connections.pr
Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 8.0.39
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-07-01T19:07:40.214+05:30 INFO 11868 --- [orm-learn] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitia
2026-07-01T19:07:40.258+05:30 INFO 11868 --- [orm-learn] [ restartedMain] j.LocalContainerEntityManagerFac
2026-07-01T19:07:40.700+05:30 WARN 11868 --- [orm-learn] [ restartedMain] JpaBaseConfiguration$JpaWebConf:
2026-07-01T19:07:41.278+05:30 INFO 11868 --- [orm-learn] [ restartedMain] o.s.b.d.a.OptionalLiveReloadSer
2026-07-01T19:07:41.339+05:30 INFO 11868 --- [orm-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatW
2026-07-01T19:07:41.355+05:30 INFO 11868 --- [orm-learn] [ restartedMain] c.c.orm_learn.OrmLearnApplicati
2026-07-01T19:07:41.362+05:30 INFO 11868 --- [orm-learn] [ restartedMain] c.c.orm_learn.OrmLearnApplicati
2026-07-01T19:07:41.648+05:30 DEBUG 11868 --- [orm-learn] [ restartedMain] org.hibernate.SQL
select
  c1_0.code,
  c1_0.name
from
  country c1_0
Hibernate:
select
  c1_0.code,
  c1_0.name
from
  country c1_0
2026-07-01T19:07:41.692+05:30 DEBUG 11868 --- [orm-learn] [ restartedMain] c.c.orm_learn.OrmLearnApplicati
2026-07-01T19:07:41.693+05:30 INFO 11868 --- [orm-learn] [ restartedMain] c.c.orm_learn.OrmLearnApplicati
```

```
or      : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA
toryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
guration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed d
er       : LiveReload server is running on port 35729
oServer  : Tomcat started on port 8081 (http) with context path '/'
n        : Started OrmLearnApplication in 5.939 seconds (process running for 6.417)
n        : Start
        :

n        : Countries = [Country [code=IN, name=India], Country [code=US, name=United States]]
n        : End
```

```
        : Countries = [Country [code=IN, name=India], Country [code=US, name=United States]]
        : End
```

Difference between JPA, Hibernate and Spring Data JPA

Aim

To study the difference between JPA, Hibernate and Spring Data JPA.

Java Persistence API (JPA)

JPA stands for **Java Persistence API**.

It is a **specification** that gives rules for connecting Java applications with a database.

JPA only tells **what should be done**, but it does not tell **how to do it**. So, it cannot work by itself. We need an implementation like Hibernate.

Points

- JPA is a specification.
- It contains only rules.
- It does not have implementation.
- It is used for Object Relational Mapping (ORM).

Hibernate

Hibernate is an **ORM framework**.

It follows the rules given by JPA and provides the implementation.

Hibernate converts Java objects into database records and database records into Java objects automatically. Because of this, we don't have to write SQL queries for every operation.

Points

- Hibernate is an implementation of JPA.
- It is an ORM framework.
- It reduces SQL coding.
- It supports different databases like MySQL and Oracle.

Spring Data JPA

Spring Data JPA is built on top of Hibernate.

It makes database programming much easier by reducing the amount of code we write.

Instead of writing session and transaction code, we can simply use methods like **save()**, **findAll()**, **findById()** and **deleteById()**.

Points

- Spring Data JPA uses Hibernate internally.
- It reduces boilerplate code.
- It provides ready-made CRUD methods.
- It is easy to use.

Difference between JPA, Hibernate and Spring Data JPA

JPA	Hibernate	Spring Data JPA
It is a specification.	It is an ORM framework.	It is built on top of Hibernate.
It contains only rules.	It implements JPA.	It reduces coding.
No implementation.	Provides implementation.	Uses Hibernate internally.
Cannot work alone.	Can work independently.	Makes database operations easier.

Hibernate Code

```
Session session = factory.openSession();  
Transaction tx = session.beginTransaction();
```

```
session.save(employee);
```

```
tx.commit();
```

```
session.close();
```

Explanation

In Hibernate, we have to:

- Open the session.
- Start the transaction.
- Save the object.
- Commit the transaction.
- Close the session.

So, Hibernate requires more code.

Spring Data JPA Code

```
public interface EmployeeRepository extends  
JpaRepository<Employee, Integer> {  
  
}
```

```
@Autowired  
private EmployeeRepository employeeRepository;
```

```
@Transactional  
public void addEmployee(Employee employee) {  
  
    employeeRepository.save(employee);  
  
}
```

Explanation

In Spring Data JPA, we simply use

```
employeeRepository.save(employee);
```

Spring automatically opens the session, starts the transaction, saves the data and closes the session.

So, the code is shorter and easier.

Conclusion

JPA gives only the rules.

Hibernate follows those rules and provides the implementation.

Spring Data JPA uses Hibernate and reduces the amount of code we write.

Therefore, Spring Data JPA is easier and faster to develop applications.

